Powering FloodNet: Optimizing Sensor Reporting for Battery Life

15-Minute Reporting Cuts Sensor Energy Use 41% and Extends Battery Life from 49 to 84 Days

Nicholas Reinoso,[1] Charlie Mydlarz,[2] Andrea I. Silverman[2,3]

1. Undergraduate Summer Research Program, NYU Tandon School of Engineering 2. Center for Urban Science and Progress, NYU Tandon 3. Department of Civil, Urban, and Environmental Engineering, NYU Tandon

Project Supervisors: Charlie Mydlarz and Andrea I. Silverman

Acknowledgments: The author thanks the NYU Tandon School of Engineering's Office of Undergraduate Academics for generous funding of this project. Thanks also to Praneeth Challagonda, Chris Hoyte, Amanpreet Kaur, Polly Pierone, and Bea Steers.

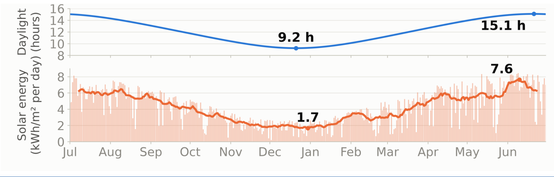
Abstract

FloodNet is a street-level flood monitoring network with 450 sensors across New York City's five boroughs. Each solar-powered unit measures water level every minute by ultrasonic sensor and reports over cellular at a remotely settable reporting interval (default 1 minute). In winter, short days and building shading cut solar harvest below what a shaded sensor consumes. Batteries run down over weeks and sensors shut off and stop reporting. A dead sensor needs a two-person crew to drive out with a ladder and a charged replacement, and a far-site swap in this citywide network can take the better part of a day. To measure how much energy each reporting interval actually consumes, a FloodNet sensor's current draw was recorded with a Qoitech Otii Arc power analyzer at 1-, 15-, and 30-minute intervals (setup below). Slowing reporting from 1 to 15 minutes cut average draw by 41%, from 2.82 to 1.66 mA, stretching a full charge of its Samsung INR18650-35E cell (3,350 mAh) from 49 to 84 days with no solar input (Figure 2). Slowing to 30 minutes saved nothing more: a 900-second MQTT keepalive wakes the radio mid-interval anyway. Fifteen-minute reporting delivers the most live data at the lowest measured power, at the price of reports up to 15 minutes late. A server-side policy is proposed: 15-minute reporting below a battery voltage threshold, with 1-minute reporting restored when storms or nearby flooding threaten.

Testing

Testing used a Qoitech Otii Arc power analyzer recording current at 4,000 samples per second at the battery's 3.60 V nominal voltage. The 1-, 15-, and 30-minute reporting intervals were captured for 63, 23, and 16 hours, respectively. The Arc's serial (UART) log matched each current spike to a logged sensor event.

Figure 1: Daylight and daily solar energy (kWh/m² per day) near NYC, July 2025 to June 2026 (solar data: Open-Meteo, CC BY 4.0)



Results

Figure 2: Predicted state of charge at 1-, 15-, and 30-minute reporting on the Samsung INR18650-35E cell, no solar input

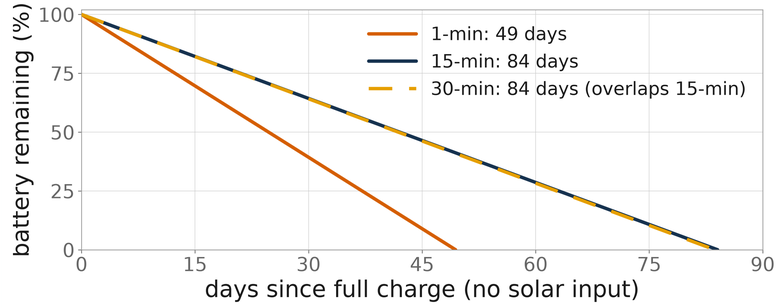


Figure 3: Radio duty cycle at each reporting interval: the share of each hour the radio is on (dark pulses are uploads, orange are keepalive pings)

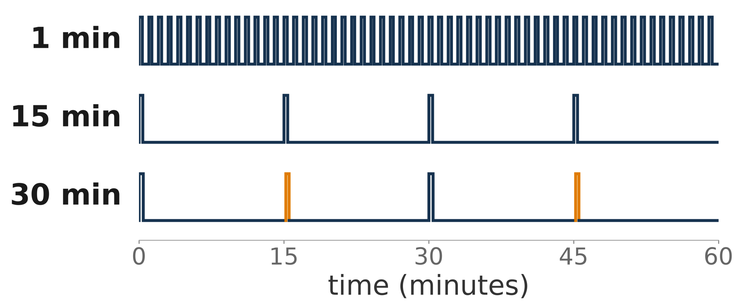


Figure 3 shows the fraction of each hour the radio is on. Radio-on time is what consumes the battery, and it is nearly identical at 15- and 30-minute reporting intervals, which is why 30-minute reporting saves nothing more.

The averages behind Figures 2 and 3 come from deliberately long captures: 63, 23, and 16 hours at the three reporting intervals. Averaging over dozens of reporting cycles and many hours of cellular network variability makes the averages stable rather than dependent on any single stretch of conditions, computed over complete hours only with the first boot hour excluded.

The radio wakes only once per reporting interval to send its stored readings, but the MQTT connection times out after 900 seconds without traffic, so the 30-minute reporting interval adds a mid-interval keepalive ping (orange pulses, Figure 3).

Figure 4: Radio wakes per hour at 15- and 30-minute reporting

Because of the keepalive ping (Figure 4), 15- and 30-minute reporting produce the same four radio wakes per hour and nearly identical radio-active time, so their energy use is equivalent (Figure 3). Radio wakes draw the sensor's highest instantaneous current (transmit peaks above 100 mA against a 1.26 mA idle floor). The added keepalive wake cancels the saving that halving the upload rate should provide.

The cost of slower reporting is latency: a report of street flooding can arrive up to the reporting interval late. Proposal (1) below counters this by restoring 1-minute reporting when a storm is forecast or nearby flooding is detected.

Future Work

Proposals: (1) a server-side policy: 1-minute reporting above a battery voltage threshold, 15-minute below, with 1-minute reporting restored when storms or nearby flooding threaten; (2) extending the MQTT keepalive from 15 minutes (900 s) to 19 minutes (1,140 s), just under the Azure IoT Hub limit, and measuring energy at the longer setting; (3) shortening the 250 ms pause the firmware leaves between queued records during an upload.